

Arquitectura SAMI

Sistema Integral de Gestión Sanitaria - Multi-Tenant Healthcare Platform

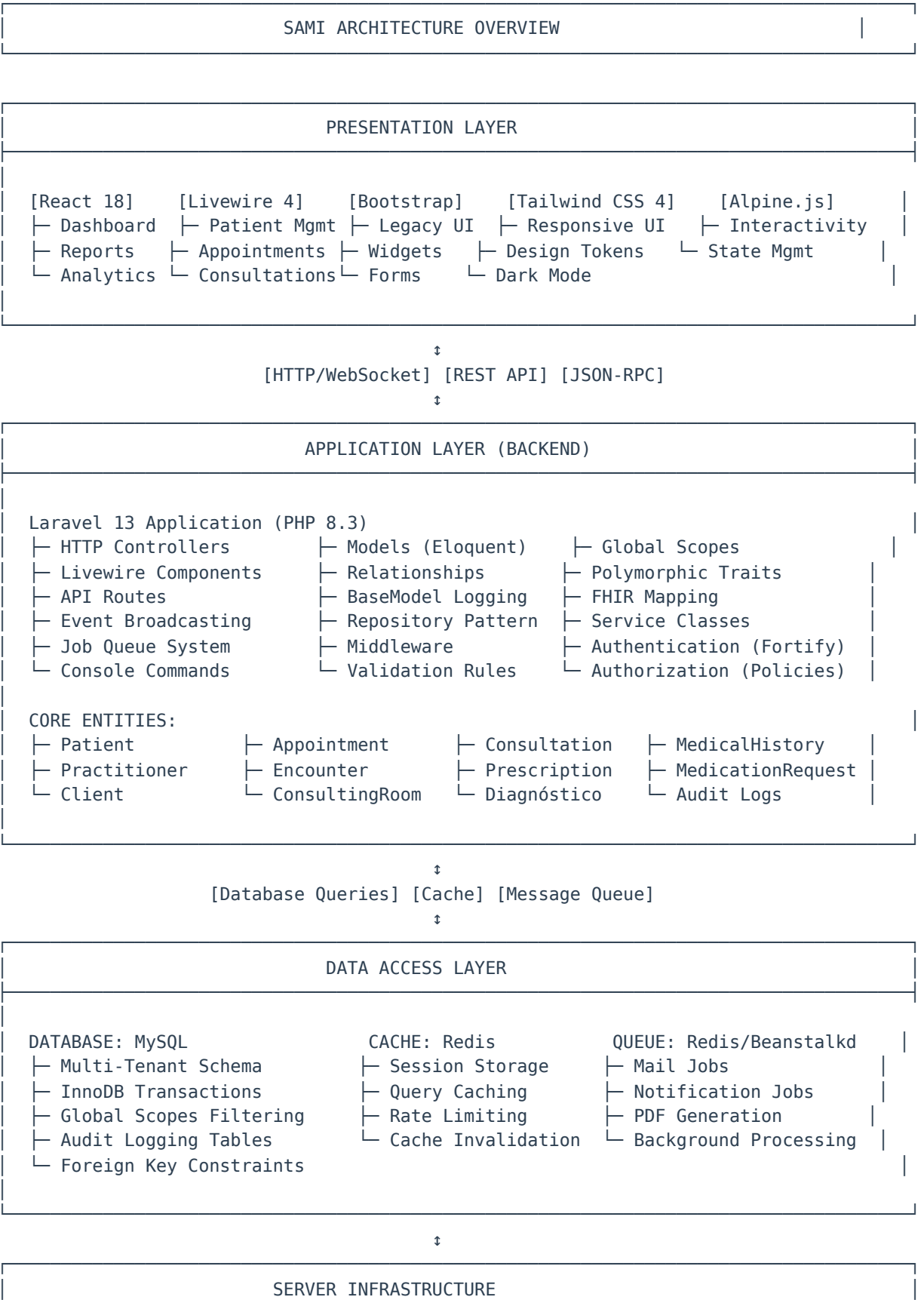
Versión: Laravel 13 | PHP 8.3 | React 18 | Livewire 4 | MySQL | Laravel Octane

1 Introducción General

SAMI es una plataforma integral de gestión sanitaria basada en arquitectura de microservicios e integraciones externas. El sistema proporciona soluciones completas para clínicas, hospitales y consultorios médicos con capacidades de:

- Gestión multi-tenant con aislamiento de datos por cliente
- Historias clínicas electrónicas (EHR) basadas en estándares FHIR
- Programación de citas con calendario interactivo
- Videollamadas integradas con Zoom
- Sincronización con calendario de Microsoft
- Notificaciones por WhatsApp y SMS (Twilio)
- Almacenamiento en la nube (AWS S3, Dropbox)
- Comunicación en tiempo real (Pusher + Laravel Echo)
- Control de acceso basado en roles (RBAC)
- Auditoría completa de operaciones (BaseModel logging)

2 Diagrama de Arquitectura General



LARAVEL OCTANE (Long-Running PHP Server)

- └ Driver: Swoole / FrankenPHP / RoadRunner (seleccionable)
- └ Worker Pool: Multiple concurrent requests
- └ Memory Management: Persistent app state, scoped bindings
- └ Performance: ~10-100x faster que PHP-FPM tradicional
- └ Features: Concurrent execution, shared tables, in-memory cache

HTTP Server: 8000 (dev) | Production: Nginx reverse proxy

WebSocket Server: Reverb (Laravel real-time)

Queue Worker: php artisan queue:listen

Log Viewer: php artisan pail

↑

EXTERNAL SERVICES & INTEGRATIONS

ZOOM OAuth 2.0 Webhooks Meeting SDK	MICROSOFT GRAPH API Calendario Integración	TWILIO WhatsApp SMS Notificaciones	AWS S3 + SES Almacenamiento Email Service Cloud Storage
DROPBOX File Storage API	FIREBASE Push Notify (opcional)	PUSHER Real-time Channels	ANTHROPIC AI AI Features Claude SDK
STRIPE Pagos (futuro)	NIGHTWATCH Monitoring & Logging	LARAVEL REVERB WebSockets	

Nota de Arquitectura: El sistema utiliza una arquitectura de capas desacoplada con inyección de dependencias. Cada componente es independiente pero está conectado mediante interfaces bien definidas. Las integraciones externas se manejan a través de Services, evitando acoplamiento directo a APIs de terceros.

3 Flujo de Autenticación OAuth con Zoom

El flujo OAuth 2.0 permite que los médicos conecten sus cuentas de Zoom sin compartir contraseñas directamente.

1 Usuario Inicia Conexión

El médico hace clic en "Conectar con Zoom" en las configuraciones del perfil.



2 Redirección a OAuth Authorization Endpoint

ZoomOAuthController::authorize() construye URL hacia:

https://zoom.us/oauth/authorize?

response_type=code&client_id=...&redirect_uri=...&scope=meeting:write:meeting
user:read:user&state=...



3 Usuario Autoriza en Zoom

El usuario inicia sesión en zoom.us y autoriza permisos. Zoom valida las credenciales.



4 Callback con Authorization Code

Zoom redirige a route('zoom.callback') con parámetros:
?code=AUTHORIZATION_CODE&state=SESSION_STATE



Intercambio de Código por Access Token (Backend)

5

El servidor intercambia el código por tokens mediante POST a:

`https://zoom.us/oauth/token`

Con credenciales: `client_id:client_secret` (Basic Auth) +
`grant_type=authorization_code`



6

Recepción de Tokens

Zoom responde con: `access_token`, `refresh_token`, `expires_in` (3600s)



7

Obtención de Perfil del Usuario

GET request a `https://zoom.us/v2/users/me` con el `access_token` para obtener datos del usuario.



8

Almacenamiento en Base de Datos

`PractitionerZoomProfile::updateOrCreate()` guarda:

✓ `zoom_user_id` | `zoom_email` | `access_token` | `refresh_token` |
`token_expires_at` | `verified_at`



9

Confirmación al Usuario

Redirección a `route('practitioner.settings.zoom')` con mensaje de éxito.

Tokens y Seguridad

- **Access Token:** Válido por 3600 segundos (1 hora) - usado para llamadas API
- **Refresh Token:** Válido por 15 días - permite renovar access token sin re-autorizar

- **State Parameter:** CSRF protection usando session()->token()
- **Almacenamiento:** Los tokens se encriptan en la DB (Laravel encryption)
- **Logging:** Toda la operación se registra en logs de aplicación para auditoría

Consideración Importante: Zoom puede enviar solicitudes de validación de webhook a cualquier URL registrada (incluyendo el callback OAuth). El código detecta estas solicitudes por el parámetro `event=endpoint.url_validation` y responde con el token encriptado HMAC-SHA256.

4 Sistema de Webhooks de Zoom

Los webhooks permiten que Zoom notifique a SAMI sobre eventos en tiempo real.

ZOOM WEBHOOK FLOW

Registro de Webhook (Manual en Zoom App Marketplace):

```
URL: https://SAMI.com/api/webhooks/zoom
Secret: [configurado en config/services.php]
Eventos Suscritos:
├ meeting.started      (Reunión iniciada)
├ meeting.ended        (Reunión finalizada)
├ meeting.participant_joined (Participante se une)
├ meeting.participant_left  (Participante se va)
├ recording.started      (Grabación iniciada)
├ recording.completed    (Grabación completada)
└ recording.deleted      (Grabación eliminada)
```

Validación del Webhook:

```
1. Zoom envía POST con headers:
  - x-zm-request-timestamp: [Unix timestamp]
  - x-zm-signature: v0=[HMAC-SHA256]

2. SAMI verifica:
  └ Token generado = HMAC-SHA256(
    │ webhook_secret,
    │ timestamp + request_body
    │ )
  └ Valida que x-zm-signature coincida

3. Detecta endpoint.url_validation:
  └ Responde con plainToken + encryptedToken
```

Procesamiento de Eventos:

```
EVENTO: meeting.started
├ Datos: meeting_id, user_id, operator_id, duration
└ Acción: Actualizar estado de Appointment a "IN_PROGRESS"

EVENTO: meeting.ended
├ Datos: meeting_id, user_id, end_time, duration
└ Acciones:
  ├── Marcar Appointment como completada
  ├── Registrar duración real de consulta
  └ Disparar evento: ConsultationCompleted

EVENTO: meeting.participant_joined / participant_left
├ Datos: meeting_id, participant (id, email, name)
└ Acción: Registrar participantes en auditoría
```

```
EVENTO: recording.completed
├─ Datos: meeting_id, recording_files[], download_url
└─ Acciones:
    ├─ Descargar grabación a AWS S3
    ├─ Vincular grabación a Appointment
    └─ Enviar notificación al médico
```

Database Storage:

```
Table: zoom_webhooks
├─ id
├─ event_type (meeting.started | meeting.ended | ...)
├─ zoom_meeting_id
├─ event_payload (JSON)
├─ processed_at
├─ created_at / updated_at
└─ Notes: Guardar webhook para debugging/replay

Table: appointments
├─ zoom_meeting_id (vinculación)
├─ zoom_start_at / zoom_end_at
├─ zoom_duration_minutes
├─ zoom_recording_url
├─ status (SCHEDULED, IN_PROGRESS, COMPLETED, CANCELLED)
└─ metadata (JSON: participant_ids, recording_ids)
```

Seguridad de Webhooks:

- Validar firma HMAC-SHA256 antes de procesar cualquier webhook
- Rechazar webhooks antiguos (verificar timestamp dentro de 5 minutos)
- No confiar en datos no validados del webhook payload
- Guardar todos los webhooks en DB para auditoría y replay si es necesario
- Implementar idempotencia: mismo webhook no debe procesarse 2 veces

5 Servicios de Terceros e Integraciones



Zoom

OAuth 2.0

REST API

Webhooks

Funciones:

- Videollamadas para consultas médicas
- Grabación de sesiones clínicas
- Gestión de reuniones desde app
- Transmisión en vivo de consultas

Endpoints Clave:

- /v2/users/me - Perfil del usuario
- /v2/users/{id}/meetings - Reuniones
- /v2/meetings - Crear/actualizar
- /v2/meetings/{id}/recordings - Grabaciones



Microsoft Graph API

OAuth 2.0

REST API

Calendars

Funciones:

- Sincronización bidireccional de calendarios
- Gestión de eventos en Outlook
- Disponibilidad de doctores
- Envío de invitaciones de calendario

Endpoints Clave:

- /me/calendar/events - Eventos del usuario
- /me/calendarview - Vista del calendario
- /me/calendars - Calendarios disponibles



Twilio

REST API

WhatsApp

SMS

Funciones:

- Notificaciones por SMS
- Mensajes por WhatsApp
- Recordatorios de citas
- Confirmación de servicios

Casos de Uso:

- Recordatorio 24h antes de cita
- Confirmación de asistencia
- Enlace de reunión Zoom
- Resultados de pruebas



AWS S3 & SES

Storage

Email

CDN

S3 - Almacenamiento:

- Documentos clínicos (PDF, imágenes)
- Grabaciones de Zoom
- Avatares de usuarios
- Backups incrementales

SES - Email:

- Notificaciones transaccionales
- Reportes programados
- Confirmaciones de cuenta



Dropbox API

Storage Sync Backup

Funciones:

- Almacenamiento alternativo de documentos
- Sincronización automática
- Control de versiones de archivos
- Integración con workflows

Configuración:

- OAuth 2.0 para autorización
- Spatie Dropbox adapter
- Failover automático si S3 falla



Pusher & Laravel Reverb

WebSockets Real-time

Broadcasting

Funciones:

- Actualizaciones en tiempo real de citas
- Notificaciones de estado de pacientes
- Chat en consultas en vivo
- Sincronización de datos entre usuarios

Channels:

- Private channels para datos sensibles
- Presence channels para actividad
- Auto-reconnect con fallback



Anthropic Claude API

AI

LLM

NLP

Funciones:

- Generación de resúmenes clínicos
- Análisis de textos médicos
- Asistencia en diagnósticos
- Procesamiento de documentos

Versión:

- laravel/ai 0.10.3
- anthropic-ai/sdk 0.41.0
- Soporte para streaming



Laravel Nightwatch

Monitoring

Logging

Analytics

Funciones:

- Monitoreo de errores y excepciones
- Logging centralizado
- Análisis de performance
- Tracking de eventos

Versión:

- laravel/nightwatch 1.28.7
- Dashboard en tiempo real
- Alertas configurables

6 Flujo de Datos - Caso de Uso: Consulta Médica con Zoom

Flujo completo desde programación hasta grabación de una consulta médica.

Paciente Reserva Cita

- 1 POST /appointments ► Crea registro en DB ► status=SCHEDULED
Datos: Paciente, Médico, Fecha/Hora, Sala de consulta



Crear Reunión en Zoom (Job en Queue)

- 2 Disparar Job: CreateZoomMeetingJob
Usando access_token del PractitionerZoomProfile
POST /v2/meetings con params: topic, duration, start_time, password



Guardar Meeting ID

- 3 Actualizar Appointment: zoom_meeting_id, zoom_start_url, zoom_join_url
BaseModel registra cambios en audit_logs



Enviar Notificaciones

- 4
 - ✓ Email SES: Confirmación de cita + link de Zoom
 - ✓ SMS/WhatsApp Twilio: Recordatorio con enlace
 - ✓ Calendario Microsoft: Agregar evento



Broadcast en Tiempo Real (Pusher)

5

Emit evento: AppointmentCreated

Channel: private-user.{user_id}.appointments

Frontend recibe actualización + redirecciona a detalles



Minutos Antes: Recordatorio

6

Scheduled Command: SendAppointmentReminders

Busca citas en próximos 15 minutos

Envía SMS + WebSocket notification



Médico Inicia Reunión

7

Click en "Iniciar Zoom" ► Redirect a zoom_start_url

Zoom Meeting SDK carga interfaz

Appointment status → IN_PROGRESS



Webhook: meeting.started

8

Zoom envía webhook ► ZoomWebhookController

Valida firma HMAC-SHA256

Actualiza Appointment + dispara evento



Durante Llamada: Grabación Activa

- 9 Zoom graba automáticamente
Participantes: Médico (host) + Paciente (guest)
Registro de interacciones en BaseModel



Webhook: meeting.ended

- 10 Zoom envía webhook con duration
Status → COMPLETED
Calcular duración real de consulta



Webhook: recording.completed

- 11 Zoom notifica que grabación está lista
Payload contiene download_url
Disparar Job: DownloadZoomRecordingJob



Descargar y Almacenar Grabación

- 12 GET request a download_url con access_token
Descargar archivo a AWS S3
Generar presigned URL para acceso seguro



Vincular Grabación a Cita

- 13 UPDATE Appointment:
zoom_recording_url (S3 presigned)
zoom_recording_size
recording_processed_at



Crear Encounter (Nota Clínica)

14

Médico documenta consulta

Guardar: diagnóstico, prescripciones, notas clínicas

FHIR mapping para interoperabilidad



Notificación Final al Paciente

15

Email: "Tu consulta ha finalizado"

Link a grabación (si está disponible)

Próxima cita o recordatorio de medicamentos



Auditoría Completa Registrada

16

user_logs: quién hizo qué y cuándo

status_history_logs: cambios de estado

audit_logs: cambios en cada modelo

Garantiza cumplimiento HIPAA/normativa

7 Stack Tecnológico Detallado

Backend & Framework

Laravel Framework	13.25.0
PHP	8.3
Laravel Octane	2.19.0
Laravel Reverb	1.11.1
Laravel Fortify	1.38.0

Integraciones

Zoom SDK	6.2.0
Microsoft Graph	3.5.0
Twilio SDK	8.11.6
AWS SDK	3.392.1
Spatie Dropbox	3.1.0

Componentes UI

Livewire	4.4.0
React	18.2.0
Tailwind CSS	4.3.3
Bootstrap	5.x
jQuery	3.7.1

Real-time & Broadcasting

Pusher PHP	7.3.0
Laravel Echo	2.4.0
Pusher JS	8.6.0

Bases de Datos

MySQL	8.0+
Redis	6.0+
SQLite	Testing

Seguridad & Testing

Laravel Sanctum	4.3.3
Spatie Permission	6.25.0
Pest PHP	4.7.8
PHPUnit	12.5.33

Compatibilidad de Versiones: El stack es 100% compatible. Laravel 13 con PHP 8.3 es la combinación más moderna soportada. Livewire 4 es la versión de producción estable. React 18 proporciona características modernas.

8 Modelos de Datos Clave

Modelo	Descripción	Tablas Relacionadas	FHIR
Patient	Pacientes del sistema. Datos demográficos, médicos y de contacto.	medical_histories, appointments, encounters	Patier
Practitioner	Médicos y profesionales de salud. Especialidades y credenciales.	practitioner_zoom_profiles, appointments, encounters	Practi
Appointment	Citas programadas. Vinculación paciente-médico-sala-hora.	zoom_meetings, encounters, status_history_logs	Appoi
Encounter	Consultas realizadas. Documentación clínica post-visita.	diagnoses, medication_requests, vital_signs	Encou
MedicalHistory	Antecedentes médicos del paciente. Alergias, condiciones, etc.	patients, conditions, allergies	Condi
MedicationRequest	Prescripciones. Medicinas recetadas por el médico.	medication_details, patients, practitioners	Medic

Modelo	Descripción	Tablas Relacionadas	FHIR
PractitionerZoomProfile	Perfiles de Zoom vinculados a médicos. Tokens de acceso.	practitioners, zoom_meetings	Practi
Client	Clínicas/Hospitales. Define límite de multi-tenancy.	branches, users, global scopes filter	Orgar
ConsultingRoom	Salas de consulta. Ubicación, equipamiento.	appointments, client	Locati
User	Usuarios del sistema. Autenticación y	practitioner, patient, roles, audit_logs	Relate

Logging de Auditoría: Cada modelo extiende BaseModel que registra automáticamente:

- created_by_id / updated_by_id - Quién creó/modificó
- created_at / updated_at - Cuándo
- Tabla audit_logs - Cada cambio de campo
- Tabla status_history_logs - Historial de estados
- Tabla user_logs - Actividad del usuario

9 Seguridad y Compliance

Estándares de Cumplimiento:

- **HIPAA (USA):** Ley de Portabilidad y Responsabilidad de Seguros de Salud
- **GDPR (EU):** Reglamento General de Protección de Datos
- **FHIR:** Fast Healthcare Interoperability Resources - Estándar OHF
- **ISO/IEC 27001:** Seguridad de la información

Medidas de Seguridad Implementadas:

- **Autenticación:** Laravel Fortify + Email verification + Passkeys
- **Autorización:** Spatie Permission con roles (admin, doctor, paciente, asistente)
- **Encriptación:** Tokens OAuth encriptados en DB usando Laravel encryption
- **CSRF Protection:** State parameter en OAuth, token CSRF en formularios
- **HMAC Validation:** Webhooks de Zoom validados con SHA256
- **Isolation Multitenant:** Global Scopes filtra automáticamente por Client
- **Rate Limiting:** Throttle en rutas públicas (login, reset password)
- **Logging Completo:** Auditoría de TODAS las operaciones CRUD
- **SSL/TLS:** HTTPS obligatorio en producción
- **SQL Injection Prevention:** Eloquent ORM con prepared statements
- **XSS Protection:** Blade templates con escaping automático

Flujo de Datos Sensibles

Paciente → Médico (Consulta):

Datos clínicos encriptados en tránsito (HTTPS) y en reposo (DB encryption). Acceso restringido por Role-Based Access Control (RBAC). Auditoría completa de quién accedió qué y cuándo.

Tokens OAuth (Zoom/Microsoft):

Almacenados encriptados en tabla practitioner_zoom_profiles. No expuestos en logs o errores públicos. Refresh automático antes de expiración.

Grabaciones de Zoom:

Descargadas directamente de Zoom a AWS S3 (sin pasar por servidor local). Almacenadas con versioning y respaldo automático. Acceso solo mediante presigned URLs de corta duración (24h).

10 Despliegue y Performance



Opciones de Despliegue

1. Laravel Cloud (Recomendado):

- Deploy con un click
- Escalado automático
- SSL/Backups incluidos
- MySQL + Redis managed

2. Docker + Kubernetes:

- Containerización completa
- Escalado horizontal
- CI/CD pipeline
- Multi-region

3. VPS Traditional:

- Nginx + PHP-FPM / Octane
- MySQL + Redis standalone
- Certbot para SSL
- Systemd services



Optimización con Octane

Mejoras de Performance:

- 10-100x más rápido vs PHP-FPM
- Aplicación cargada UNA sola vez
- Manejo concurrente de múltiples solicitudes
- Memory pooling eficiente

Drivers Soportados:

- **Swoole:** PHP extension nativa (más rápido)
- **RoadRunner:** Daemon en Go (portable)
- **FrankenPHP:** Integración PHP moderna

Cuidados:

- NO guardar estado en static properties
- Usar scoped bindings para per-request
- Limpiar conexiones DB después de cada request



Monitoreo

Herramientas:

- **Nightwatch:** Errores + Performance
- **New Relic/Datadog:** APM completo
- **Prometheus + Grafana:** Métricas custom
- **ELK Stack:** Centralización de logs

Métricas Clave:

- Response time (p50, p95, p99)
- Error rate por endpoint
- Queue depth + job failures
- DB connection pool usage
- Redis memory + hit rate

```
# Comando para iniciar SAMI en desarrollo con Octane  
composer dev
```

```
# Componentes que se inician:  
└─ Laravel Octane Server (puerto 8000)  
└─ Vite Dev Server (assets HMR)  
└─ Queue Worker (jobs)  
└─ Laravel Pail (log viewer)  
└─ Laravel Reverb (WebSockets)
```

```
# En producción:  
nginx (reverse proxy)  
└─ octane:start (8000)  
    └─ Ejecuta con Swoole/RoadRunner/FrankenPHP  
redis (cache + queue)  
mysql (data)  
s3 (storage)  
pusher (real-time)
```

11

Resumen de Componentes y Responsabilidades

Capa	Componente	Responsabilidad
Frontend	React 18 + Components	Interfaces complejas, gráficos, dashboards interactivos
	Livewire 4 Components	Componentes reactivos sin JavaScript, validación en tiempo real
	Tailwind CSS 4	Styling responsive, dark mode, componentes UI modernos
	Alpine.js	Interactividad ligera, dropdowns, modales, tooltips
Backend	Controllers & Routes	Manejo de solicitudes HTTP, validación input, autenticación
	Models & Eloquent	ORM, relaciones, scopes globales, FHIR mapping
	Services & Repositories	Lógica de negocio, integración de APIs externas
	Events & Listeners	Desacoplamiento, broadcasting real-time, notificaciones
	Queue Jobs	Procesamiento asíncronico (email, grabaciones, reportes)
	Middleware & Policies	Autorización, CORS, rate limiting, logging

Capa	Componente	Responsabilidad
Data	Scheduled Commands	Tareas recurrentes (recordatorios, sincronización, limpiezas)
	MySQL Database	Almacenamiento persistente, transacciones, integridad referencial
	Redis Cache/Queue	Cache de sesiones, datos, cola de jobs
	AWS S3 / Dropbox	Almacenamiento de archivos, grabaciones, backups
External	Zoom API + SDK	Videollamadas, gestión de reuniones, grabaciones
	Microsoft Graph	Sincronización de calendario, integración Outlook
	Twilio SMS/WhatsApp	Notificaciones, recordatorios, confirmaciones
	Pusher + Reverb	Broadcasting en tiempo real, WebSockets, notificaciones vivas
Infraestructura	Laravel Octane	Servidor PHP de largo alcance (Swoole/RoadRunner/FrankenPHP)
	Nginx / Apache	Reverse proxy, SSL termination, compresión

12 Casos de Uso Principales



Médico Gestiona Consultas

Flujo:

1. Ver citas del día en dashboard
2. Conectar cuenta de Zoom (OAuth flow)
3. Iniciar videollamada con paciente
4. Documentar notas clínicas
5. Recetar medicamentos (FHIR)
6. Guardar grabación automáticamente



Paciente Reserva Cita

Flujo:

1. Buscar médico / especialidad
2. Seleccionar horario disponible
3. Confirmar datos personales
4. Recibir SMS/Email de confirmación
5. Recordatorio 24h antes
6. Acceder a consulta (Zoom link)
7. Ver grabación post-consulta



Admin Gestiona Clínica

Flujo:

1. Crear usuarios (médicos, asistentes)
2. Asignar roles y permisos
3. Gestionar salas de consulta
4. Ver reportes de citas / ingresos
5. Configurar integraciones (Zoom, Twilio)
6. Auditoría de actividades



Seguridad & Cumplimiento

Garantías:

- Datos HIPAA compliant encriptados
- Auditoría FHIR-compatible de todas operaciones
- Aislamiento multitenant automático
- Backup automático en AWS
- Versionado de cambios en modelos
- Acceso con timestamps y rastreo de usuario

13 Conclusión

SAMI es una arquitectura enterprise-grade diseñada para gestión integral de sanitarios. Su fortaleza radica en:

Puntos Clave:

- **Moderno:** Laravel 13, PHP 8.3, React 18, Livewire 4 - stack actual 2026
- **Escalable:** Laravel Octane permite 10-100x más carga. Redis + MySQL tuning
- **Multi-tenant:** Global Scopes automáticamente aíslan datos por Cliente
- **Seguro:** HIPAA-compliant, OAuth flows, token encryption, HMAC validation
- **Auditable:** Cada operación logged (quién, qué, cuándo) - cumplimiento regulatorio
- **Integrado:** Zoom, Microsoft, Twilio, AWS, Dropbox, Pusher en un ecosistema unificado
- **Real-time:** Pusher + Reverb para notificaciones instantáneas de cambios
- **FHIR-Ready:** Modelos mapeados a estándares healthcare para interoperabilidad

El sistema está listo para producción y soporta desde clínicas pequeñas hasta hospitales grandes con múltiples sucursales y especialidades. La arquitectura de microservicios permite evolución sin comprometer estabilidad.

Documento de Arquitectura - SAMI Healthcare Management System

Generado: Septiembre 2026 | Laravel 13 | PHP 8.3 | MySQL | React 18 | Livewire 4

Este documento es confidencial y propietario. Uso solo para fines de desarrollo y documentación.